CHANDIGARH UNIVERSITY
Discover. Learn. Empower.



OOPs CASE STUDY REPORT

MEDICORE

Hospital Emergency & Patient Management System Using C++

Course: Object-Oriented Programming (OOPs)

Programming Language: C++

Project Type: Console-Based Application

SUBMITTED BY

Student Name: Ishaan Gupta

UID: 25BCD10228

Semester: 3rd Semester

Section: 25BCD-2B

Programme: Bachelor of Computer Applications (BCA) (Data science)

SUBMITTED TO

Faculty Name: Jitendra Kumar

Department: _____

ACADEMIC YEAR: 2026–2027

1.1 Executive Summary

Healthcare management is an essential part of the modern healthcare industry. Hospitals handle numerous administrative and medical activities every day, including patient registration, doctor management, appointment scheduling, treatment documentation, emergency coordination, and maintenance of hospital records. Managing these activities requires proper organization, accuracy, and coordination between different departments.

MediCore is a proposed Hospital Emergency and Patient Management System developed using C++ and Object-Oriented Programming principles. The primary purpose of this project is to demonstrate how a computer-based application can organize basic hospital-management activities through a structured and systematic approach.

The proposed application provides a centralized, menu-driven environment through which users can perform different hospital-related operations. These include registering patients, maintaining doctor information, searching patient records, scheduling appointments, recording demonstration treatment information, simulating emergency-priority selection, and generating hospital statistics.

The system represents different hospital entities through separate classes. Patient and Doctor are represented as specialized entities derived from a common Person class. Appointment and Treatment represent separate hospital records, while the HospitalSystem class coordinates the major application operations.

The application is implemented entirely in C++. It uses standard library features and in-memory data structures to maintain information during program execution. It does not require a graphical user interface, external database, or separate backend.

This case study examines the background of the proposed hospital, the problems identified in the existing record-management process, the objectives of the proposed system, functional and non-functional requirements, feasibility, operational procedures, system architecture, module descriptions, testing strategies, expected results, limitations, and future development possibilities.

The proposed system is intended for educational purposes. Its emergency-priority mechanism is a simplified programming simulation and must not be used for actual clinical decision-making.

1.2 Purpose of the Case Study

The purpose of this case study is to analyze a hospital-management scenario and propose a structured software solution using C++.

It demonstrates how software development principles can be applied to an administrative problem and how different hospital activities can be organized into separate functional modules.

2. INTRODUCTION

2.1 Background of Healthcare Management

Healthcare institutions are responsible for providing medical services while maintaining several administrative activities. A hospital does not operate only through doctors and medical professionals; it also depends on organized administrative processes.

Patient registration, appointment scheduling, doctor allocation, treatment documentation, and record maintenance are important activities that support the functioning of a hospital.

Each activity generates information that may need to be accessed later. For example, patient registration creates personal information records, appointment scheduling creates consultation details, and treatment documentation creates information associated with a particular patient and doctor.

When these activities are managed independently, locating and coordinating information can become difficult.

A structured information-management approach can help organize these activities and make record retrieval easier.

2.2 Introduction to MediCore

MediCore is a proposed hospital-management application designed to organize selected hospital operations through a simple computer program.

The application is developed using C++ and follows an object-oriented programming approach.

Instead of maintaining all program information and operations inside one large function, MediCore divides its functionality into different classes and methods.

Each class represents a particular entity or responsibility.

For example, the Patient class maintains patient-specific information, while the Doctor class maintains doctor-related information.

Similarly, Appointment represents scheduled consultations, and Treatment represents demonstration treatment records.

The Hospital System class coordinates these components and provides access to their operations through a menu-driven interface.

2.3 Motivation Behind the Project

The main motivation behind developing MediCore is to understand how programming can be applied to practical administrative problems.

The project provides an opportunity to transform a theoretical hospital-management scenario into a structured software application.

It also helps demonstrate how C++ can be used to organize information, perform searching operations, validate user input, and manage relationships between different objects.

Another important motivation is to understand how modular software design can improve program readability and maintainability.

2.4 Importance of Computer-Based Hospital Management

A computer-based management application can help organize administrative information in a structured manner.

Its importance can be understood through the following points:

1. **Information Organization:** Different records can be maintained in separate structured collections.
2. **Record Retrieval:** Searching can be performed using identification numbers.
3. **Appointment Coordination:** Patient and doctor records can be checked before scheduling.
4. **Record Consistency:** Basic validation can reduce duplicate identifiers and invalid entries.
5. **Reporting:** Summary information can be calculated automatically.
6. **Modular Development:** Different operations can be implemented independently.

2.5 Relevance of the Project

MediCore is relevant as an educational software-development project because it connects programming concepts with a recognizable real-world problem.

It demonstrates how a hospital-management application can be divided into manageable components and developed using C++.

3. CASE STUDY BACKGROUND

3.1 Organization Profile

For this case study, a fictional healthcare institution named MediCore General Hospital is considered.

The hospital provides general medical consultations and manages patients requiring routine appointments as well as emergency assistance.

The hospital has administrative staff responsible for registering patients, maintaining doctor information, scheduling appointments, and recording relevant information.

For the purpose of this case study, the hospital is assumed to have the following operational areas:

Department	Responsibility
Administration	Overall administrative coordination
Reception	Patient registration
General Medicine	General medical consultations
Specialist Department	Specialized consultations
Emergency Department	Emergency patient coordination
Records Section	Maintenance of administrative records

These departments form the hypothetical organizational environment of the case study. The current C++ implementation does not model a complete departmental structure.

3.2 Existing Operational Environment

Initially, suppose the hospital manages its administrative activities using separate registers and manual documentation.

When a new patient visits the hospital, reception staff collects personal information and records it.

Doctor information is maintained separately, including names, specializations, and contact details.

Appointment information is also maintained independently, while treatment-related information is recorded through separate documentation.

As the number of patients increases, the hospital may experience difficulties in locating and coordinating information.

For example, if the administration wants to identify which doctor is associated with a particular appointment, the relevant information may need to be checked across multiple records.

Similarly, retrieving a patient's information may require manually searching through existing entries.

3.3 Organizational Challenges

The hypothetical hospital may experience the following difficulties:

Scattered Information: Patient and doctor information may exist in separate records.

Manual Searching: Locating a particular patient may require checking multiple entries.

Appointment Coordination: The hospital needs to verify patient and doctor information before creating appointments.

Duplicate Information: Repeated manual entries may result in inconsistencies.

Treatment Documentation: Maintaining treatment information independently can make record organization difficult.

Administrative Reporting: Calculating the number of registered patients and appointments may require manual effort.

3.4 Need for a Structured Solution

The hospital requires a structured approach to managing its selected administrative activities.

A computer-based application can organize these operations through separate modules.

MediCore is proposed to demonstrate how these operations can be represented and coordinated through a single C++ application.

4. PROBLEM IDENTIFICATION AND ANALYSIS

4.1 Problem Statement

Hospital administration involves maintaining several categories of information and coordinating different activities.

These activities include patient registration, doctor management, appointment scheduling, treatment documentation, and record retrieval.

When these activities are managed through disconnected or manual processes, locating information and coordinating records may become difficult.

The proposed MediCore application addresses this problem by providing a centralized, menu-driven environment for managing selected hospital-related records.

4.2 Detailed Problem Identification

Problem 1: Patient Record Management

Patient information must be recorded and retrieved whenever required.

Without a structured record system, searching for a particular patient may require unnecessary manual effort.

Problem 2: Doctor Information Management

Doctor information includes identification details, specialization, contact information, and availability.

Maintaining this information separately may make it difficult to associate doctors with appointments.

Problem 3: Appointment Coordination

Appointments require a relationship between a registered patient and a registered doctor.

The application must verify both records before creating an appointment.

Problem 4: Treatment Record Management

Treatment information must be associated with the correct patient and doctor.

A separate record structure can help organize this information.

Problem 5: Emergency Coordination

For educational demonstration, the system should provide a method to select a waiting patient according to a configured priority value.

Problem 6: Administrative Reporting

Hospital administration may require basic information about the number of patients, doctors, appointments, and treatment records.

Calculating these totals manually can require additional effort.

4.3 Root Cause Analysis

The identified challenges may arise because of:

- Separate record-maintenance methods.
- Repeated information entry.
- Lack of a centralized record structure.
- Limited searching mechanisms.
- Manual coordination between records.
- Manual preparation of summary information.

4.4 Impact of the Identified Problems

These challenges may affect administrative efficiency.

For example, scattered patient information can increase the time required to retrieve records.

Similarly, appointment coordination may become difficult when patient and doctor information is maintained independently.

Manual counting may also increase the effort required to prepare basic administrative summaries.

4.5 Proposed Problem Resolution

MediCore proposes to organize the selected operations within one application.

It provides separate modules for patient registration, doctor management, appointment scheduling, treatment records, emergency-priority simulation, and statistics.

The application uses object-oriented design to separate information and functionality.

This approach improves the organization of the program and provides a foundation for future enhancements.

5. OBJECTIVES OF THE CASE STUDY

5.1 Primary Objective

The primary objective is to design and implement a C++-based Hospital Emergency and Patient Management System that organizes basic hospital administrative operations using Object-Oriented Programming.

5.2 Specific Objectives

The specific objectives are:

1. To develop a menu-driven hospital-management application.
2. To register and maintain patient information.
3. To maintain doctor information and specializations.
4. To search patient records using unique identification numbers.
5. To schedule appointments between registered patients and doctors.
6. To maintain demonstration treatment records.
7. To implement a basic emergency-priority selection mechanism.
8. To generate hospital statistics.
9. To validate important user inputs.
10. To organize program functionality using separate classes and methods.
11. To demonstrate the practical application of C++.
12. To develop an understanding of modular software design.

5.3 Educational Objectives

The project also provides experience in designing classes, creating objects, managing collections, implementing constructors, validating input, and organizing program logic.

It demonstrates how inheritance, encapsulation, abstraction, and polymorphism can support the design of a larger application.

6. SCOPE OF THE PROPOSED SYSTEM

6.1 Functional Scope

The system covers six primary operational areas.

Module	Scope
Patient Management	Registration, display, and searching

Doctor Management	Registration and display
Appointment Management	Appointment creation and display
Treatment Management	Creation and display of demonstration records
Emergency Simulation	Priority-based selection of waiting patients
Hospital Statistics	Calculation of record totals

6.2 Technical Scope

The implementation is restricted to:

- C++ programming language.
- Standard C++ libraries.
- Console input and output.
- Object-oriented programming.
- In-memory vectors.
- Menu-driven navigation.

6.3 Features Outside the Scope

The current implementation does not include:

- Database integration.
- GUI development.
- Web-based access.
- Cloud storage.
- User authentication.
- Payment processing.
- Pharmacy management.
- Laboratory management.
- Clinical diagnosis.
- Real-time emergency communication.
- Clinically validated triage.

6.4 Target Users

The intended users within the hypothetical scenario include hospital administrators, reception staff, records staff, and system operators.

The current application is an educational prototype and is not intended for direct use in a real hospital.

EXISTING SYSTEM ANALYSIS

7. EXISTING SYSTEM ANALYSIS

7.1 Description of the Existing System

The existing system considered in this case study is a hypothetical manual record-management process.

Different categories of hospital information are maintained through separate registers and documents.

Patient details are recorded during registration, doctor information is maintained separately, and appointment schedules are recorded independently.

Treatment-related information is also maintained through separate documentation.

Although this method may support basic operations, retrieving connected information may require additional manual effort.

7.2 Existing System Workflow

The hypothetical workflow is:

1. A patient arrives at the hospital.
2. Reception staff collects personal information.
3. Patient details are recorded.
4. Doctor information is checked.
5. The patient is directed towards the appropriate consultation.
6. Appointment information is recorded.
7. Treatment-related information is documented.
8. Hospital administration maintains the records.

7.3 Limitations of the Existing System

No.	Limitation	Explanation
1	Manual searching	Locating records may require additional effort.
2	Separate records	Related information may exist in different locations.
3	Repeated entry	Information may be entered more than once.
4	Appointment coordination	Patient and doctor records must be checked independently.
5	Reporting	Summary information may require manual counting.

6	Record organization	Managing increasing records can become difficult.
7	Information retrieval	Finding related information may take additional time.
8	Record consistency	Separate entries may contain inconsistent information.

7.4 Need for System Improvement

The limitations identified in the hypothetical existing system create a need for a structured record-management application.

The proposed system aims to provide one menu-driven environment for selected hospital administrative operations.

7.5 Comparison of Existing and Proposed Systems

Parameter	Existing System	MediCore
Patient registration	Manual	Computer-based
Doctor records	Separate documentation	Doctor objects
Patient searching	Manual lookup	Search by ID
Appointments	Manual coordination	Appointment module
Treatment records	Separate documentation	Treatment objects
Statistics	Manual counting	Automatic counting
Record storage	Paper or separate records	In-memory vectors
Interface	Manual documentation	Console menu
Database	Not applicable	Not included

The proposed system provides structured operations but does not offer permanent storage or the full capabilities of a commercial hospital-management platform.

---

8. PROPOSED SYSTEM ANALYSIS

8.1 Overview

MediCore is proposed as a console-based application that organizes selected hospital-management activities.

The system provides a centralized menu from which users can access different functions.

Each function performs a particular operation and uses the appropriate class and collection to manage information.

8.2 Major Characteristics

Centralized Operations: Different management functions are available through one menu.

Modular Structure: Each major activity is implemented through separate methods and classes.

Basic Validation: The application checks selected invalid values and unknown record IDs.

Object-Oriented Design: Hospital entities are represented through classes and objects.

Simple Operation: Users interact through console input and output.

In-Memory Storage: Records are maintained during program execution.

8.3 Proposed System Workflow

The proposed system follows these stages:

1. Application initialization.
2. Display of the main menu.
3. Selection of a management operation.
4. Collection of required information.
5. Validation of entered information.
6. Execution of the selected operation.
7. Storage or retrieval of records.
8. Display of the result.
9. Return to the main menu.
10. Program termination when Exit is selected.

9. STAKEHOLDER ANALYSIS

9.1 Introduction

Stakeholders are individuals or groups who interact with a system or are affected by its operations.

For the MediCore case study, several hypothetical stakeholders are identified.

9.2 Stakeholder Categories

Stakeholder	Role	Main Interest
Hospital Administrator	Oversees administration	Organized records
Reception Staff	Registers patients	Easy registration

Doctors	Associated with consultations	Correct appointment information
Patients	Individuals whose information is maintained	Accurate records
Records Staff	Maintains documentation	Organized information
System Operator	Runs the application	Simple operation

9.3 Administrator Responsibilities

The administrator is responsible for monitoring the records maintained in the application.

The administrator can use the statistics module to review the total number of registered patients, doctors, appointments, and treatment records.

9.4 Reception Staff Responsibilities

Reception staff are considered the primary users for patient registration and appointment creation.

They enter patient details and select registered doctors for scheduling appointments.

9.5 Doctor-Related Responsibilities

Doctor information is maintained through the Doctor Management module.

The current application does not provide a separate doctor login or a clinical treatment workflow.

9.6 Patient-Related Responsibilities

Patients are the individuals whose information is registered and maintained in the application.

The application does not provide direct patient access or an online patient portal.

---

10. FUNCTIONAL REQUIREMENTS

Functional requirements describe the operations that the system must perform.

ID	Requirement	Description
FR-01	Patient registration	Register a new patient.
FR-02	Patient display	Display all patients.
FR-03	Patient search	Search using patient ID.
FR-04	Doctor registration	Register a doctor.

FR-05	Doctor display	Display doctor information.
FR-06	Appointment creation	Schedule an appointment.
FR-07	Appointment display	Display appointments.
FR-08	Treatment creation	Add a demonstration treatment record.
FR-09	Treatment display	Display treatment records.
FR-10	Emergency simulation	Select a waiting patient by configured priority.
FR-11	Statistics	Display record totals.
FR-12	Input validation	Reject selected invalid entries.
FR-13	Menu navigation	Allow repeated operations until exit.

10.1 Patient Registration Requirement

The application must accept patient information and create a new patient record.

It must reject duplicate patient identification numbers.

10.2 Doctor Registration Requirement

The application must accept doctor information and create a new doctor record.

It must reject duplicate doctor identification numbers.

10.3 Appointment Requirement

The application must verify that the selected patient and doctor exist.

It must also check the doctor's availability flag before creating an appointment.

10.4 Treatment Requirement

The application must verify both patient and doctor identification numbers before creating a treatment record.

10.5 Statistics Requirement

The application must calculate totals from the records stored in the respective collections.

11. NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality and operating characteristics expected from the application.

Usability: The menu should be simple and understandable.

Reliability: Basic invalid inputs should be handled without immediate program termination.

Maintainability: Separate classes and methods should make the program easier to modify.

Portability: The program should operate on systems with a compatible C++ compiler.

Performance: Basic operations should work efficiently for small in-memory datasets.

Simplicity: No external database or third-party library installation should be necessary.

Data Handling: Records should remain available during program execution.

---

12. FEASIBILITY STUDY

A feasibility study evaluates whether a proposed system is practical to develop and operate within its intended scope.

12.1 Technical Feasibility

MediCore is technically feasible because it uses standard C++ functionality and does not require complex external infrastructure.

The application can be developed using commonly available C++ compilers.

The technical requirements include:

- Understanding of C++ syntax.
- Knowledge of classes and objects.
- Understanding of inheritance.
- Knowledge of STL vectors.
- Basic input and output handling.
- Conditional statements and loops.
- Basic program-testing skills.

The project does not require a database server or web server.

12.2 Economic Feasibility

The project has minimal development costs.

C++ compilers and commonly used code editors are available without requiring specialized paid software.

The application does not require:

- Paid database hosting.
- Cloud infrastructure.
- External APIs.
- Specialized hardware.

- Commercial software licences.

Therefore, the proposed project is economically suitable as an educational mini project.

12.3 Operational Feasibility

The application uses a menu-driven interface.

Users can select the required operation by entering its corresponding number.

The program provides messages indicating whether an operation was successful or whether an error occurred.

This makes the application relatively simple to operate.

12.4 Schedule Feasibility

The project can be divided into several development activities.

Development Stage	Main Activity
Requirement Analysis	Identify hospital-management needs
System Design	Define classes and relationships
Patient Module	Implement registration and searching
Doctor Module	Implement doctor registration
Appointment Module	Implement appointment operations
Treatment Module	Implement treatment records
Emergency Module	Implement priority selection
Statistics Module	Implement reporting
Testing	Verify operations
Documentation	Prepare case study

12.5 Feasibility Conclusion

Based on the limited scope, standard technologies, and simple implementation requirements, MediCore is feasible as a C++ educational project.

---

13. SYSTEM ARCHITECTURE

13.1 Architecture Overview

MediCore follows a class-based application structure.

The Person class provides common personal information for Patient and Doctor.

Patient and Doctor represent specialized entities.

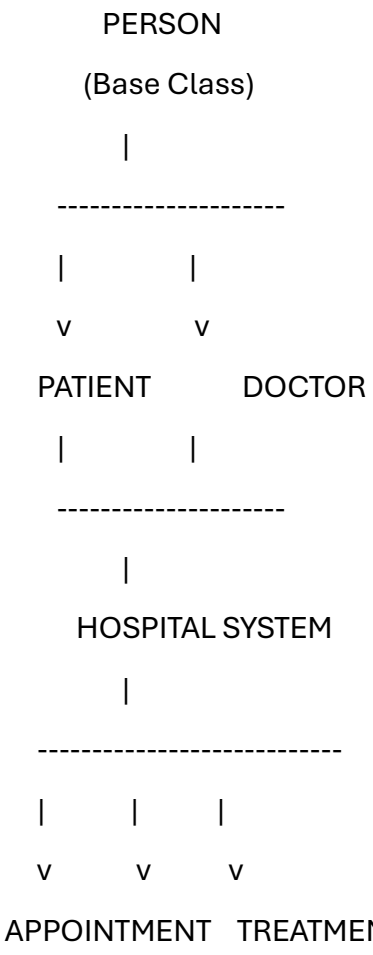
Appointment and Treatment represent different categories of records.

HospitalSystem coordinates all the main operations and maintains collections of objects.

13.2 Major System Components

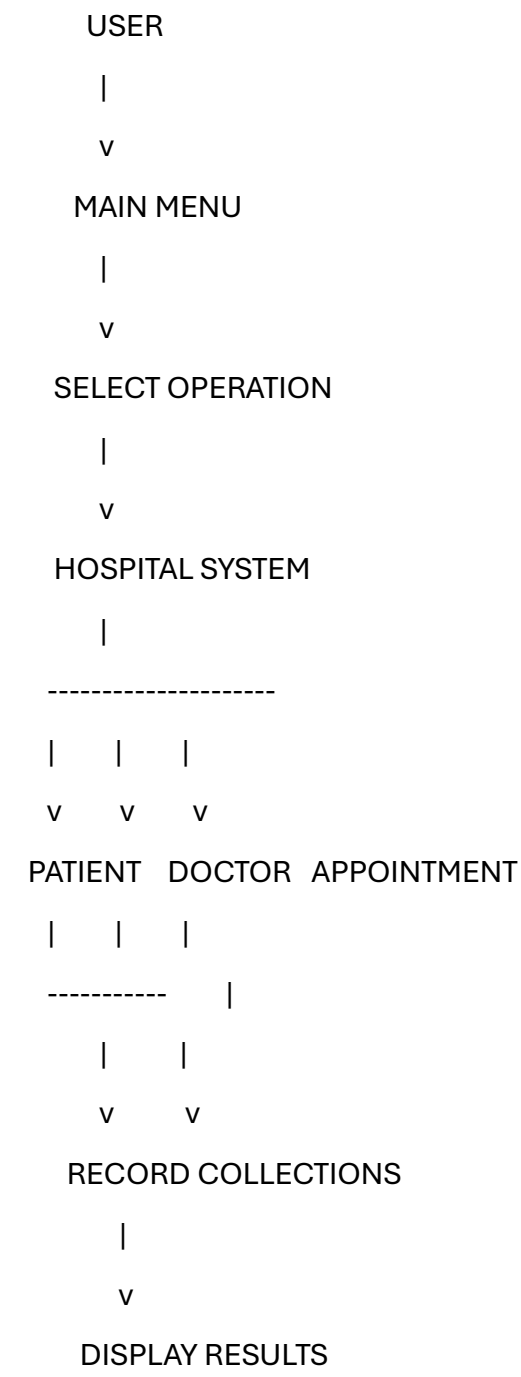
Component	Main Responsibility
Person	Common personal information
Patient	Patient-specific information
Doctor	Doctor-specific information
Appointment	Appointment details
Treatment	Demonstration treatment details
HospitalSystem	Overall application management

13.3 Conceptual Class Relationship



Patient and Doctor inherit from Person. HospitalSystem manages collections of the relevant objects.

13.4 System Data Flow



13.5 Application Control

The main function creates a HospitalSystem object and calls its menu method.

The menu method repeatedly displays available operations until the user selects Exit.

This structure separates the program's entry point from the main application functionality.

---

14. PATIENT MANAGEMENT MODULE

14.1 Introduction

Patient Management is responsible for maintaining information about individuals registered in the hospital.

Each patient is represented by an object of the Patient class.

14.2 Patient Information

Attribute	Description
Patient ID	Unique identifier
Name	Full name
Age	Patient age
Contact	Contact information
Condition	Reported condition
Severity	Configured simulation priority
Status	Current program status

14.3 Registration Procedure

When the user selects Patient Registration, the application requests the patient ID.

The entered ID is checked against the existing patient collection.

If the ID already exists, the application rejects the registration.

Otherwise, the user enters the remaining details.

The program performs basic validation and creates a Patient object if the information is acceptable.

14.4 Patient Searching

The system provides a search function based on patient ID.

The program examines the patient collection and compares the entered ID with the ID of each stored object.

If a matching record is found, the application displays the patient's information.

If no matching record exists, it displays an appropriate message.

14.5 Patient Display

The application provides an option to display all registered patients.

It traverses the patient vector and calls the displayDetails() function for each patient.

14.6 Expected Benefits

The module provides structured patient information, searching by ID, duplicate-ID checking, and consistent record display.

15. DOCTOR MANAGEMENT MODULE

15.1 Introduction

The Doctor Management module maintains information about medical professionals registered in the application.

Each doctor is represented by an object of the Doctor class.

15.2 Doctor Information

Attribute	Description
Doctor ID	Unique identifier
Name	Doctor's name
Age	Doctor's age
Contact	Contact information
Specialization	Medical specialization
Availability	Availability flag

15.3 Doctor Registration Procedure

The user enters the doctor's identification number.

The application checks whether the ID already exists.

If the ID is unique, the user enters the remaining information.

After basic validation, a new Doctor object is created and stored.

15.4 Doctor Display

The application can display all registered doctors.

The output includes the doctor's name, identification number, age, contact information, specialization, and availability.

15.5 Expected Benefits

The Doctor Management module keeps doctor information separate from patient information.

It also allows the appointment module to verify the existence and availability of a doctor.

---

16. APPOINTMENT MANAGEMENT MODULE

16.1 Introduction

The Appointment Management module connects a registered patient with a registered doctor for a scheduled consultation.

An appointment is represented by an object of the Appointment class.

16.2 Appointment Information

Attribute	Description
Appointment ID	Automatically generated ID
Patient ID	Associated patient
Doctor ID	Associated doctor
Date/Time	Scheduled date and time
Status	Appointment status

16.3 Appointment Scheduling Process

The user selects the appointment option.

The system requests the patient ID and doctor ID.

It then checks whether both records exist.

If the patient is not registered, the appointment is cancelled.

If the doctor is not registered, the appointment is also cancelled.

If both records exist and the doctor is marked available, the system requests the appointment date and time.

After the required information is entered, the application creates an appointment object.

16.4 Appointment Identification

The application generates appointment IDs using an incrementing integer.

This provides a separate identifier for each appointment created during the current session.

16.5 Appointment Display

The application provides a separate option to display all appointments.

Each record displays its appointment ID, patient ID, doctor ID, date/time, and status.

17. TREATMENT RECORD MANAGEMENT

17.1 Introduction

The Treatment Management module stores demonstration information associated with a patient and doctor.

Each record is represented by an object of the Treatment class.

17.2 Treatment Information

Attribute	Description
Patient ID	Associated patient
Doctor ID	Associated doctor
Diagnosis	Demonstration diagnosis
Notes	Demonstration treatment notes

17.3 Record Creation Procedure

The user selects Add Treatment Record.

The system requests the patient ID and doctor ID.

Both IDs are checked against the existing collections.

If either ID is invalid, the record is not created.

If both IDs are valid, the user enters diagnosis text and treatment notes.

The program creates a Treatment object and adds it to the treatment collection.

17.4 Example

Patient ID: 101

Doctor ID: 201

Diagnosis: Seasonal fever (sample)

Notes: Follow-up required (sample)

This example represents demonstration data only and must not be interpreted as medical advice.

17.5 Expected Outcome

The module provides an organized way to associate demonstration treatment records with registered patient and doctor IDs.

---

18. EMERGENCY PRIORITY SIMULATION

18.1 Introduction

The Emergency Priority Simulation module demonstrates how a program can select a record according to a configured priority value.

It uses a simple comparison algorithm to identify a waiting patient with the smallest priority number.

18.2 Priority Values

Value	Meaning in the Program
1	Highest configured priority
2	Second configured priority
3	Medium configured priority
4	Lower configured priority
5	Lowest configured priority

These are manually entered educational values and do not represent validated medical categories.

18.3 Working Procedure

The application examines the patient collection.

It first checks whether each patient's status is Waiting.

Patients with other statuses are skipped.

Among the remaining patients, the application compares the priority values.

The patient with the smallest number is selected and displayed.

18.4 Example Scenario

Consider four fictional patients:

Patient	Priority	Status
Patient A	3	Waiting
Patient B	1	Waiting
Patient C	4	Waiting
Patient D	2	Waiting

The application selects Patient B because the configured priority number is the smallest.

18.5 Expected Result

The program should correctly identify the waiting patient with the smallest configured priority value.

18.6 Safety Limitation

This module is only an educational selection algorithm.

It does not evaluate vital signs, symptoms, medical history, or validated clinical criteria.

It must not be used for actual patient triage or treatment decisions.

19. HOSPITAL STATISTICS MODULE

19.1 Introduction

The Hospital Statistics module provides a summary of the records maintained during program execution.

It helps the user understand the current record count without manually counting individual entries.

19.2 Statistics Provided

The module displays:

- Total registered patients.
- Total registered doctors.
- Total appointments.
- Total treatment records.
- Waiting patients.
- Patients with other statuses.

19.3 Calculation Method

The application uses the size of each vector to determine the number of stored objects.

It also traverses the patient collection to calculate the number of patients with different statuses.

19.4 Example

Suppose the current session contains:

Record Category	Total
Patients	3
Doctors	2
Appointments	2
Treatment Records	1

The statistics module displays the current totals.

19.5 Expected Benefits

The module provides quick access to basic administrative counts and demonstrates the use of collection traversal and counting operations.

OPERATIONAL WORKFLOW AND CASE SCENARIO

20. DETAILED OPERATIONAL WORKFLOW

20.1 Application Initialization

When the program starts, the main function creates an instance of HospitalSystem.

The system initializes its collections and displays the main menu.

20.2 Menu Selection

The user selects an operation by entering a number.

The application uses a switch statement to identify the selected function.

20.3 Information Collection

The selected module requests the required information.

For example, patient registration requests an ID, name, age, contact information, reported condition, and priority value.

20.4 Validation

The program checks important information before performing the operation.

It rejects duplicate IDs and selected invalid values.

It also checks whether required patient and doctor records exist.

20.5 Record Creation

After validation, the application creates the relevant object and adds it to the appropriate vector.

20.6 Result Display

The system displays a success message or an error message.

20.7 Repeated Operations

After completing an operation, the program returns to the main menu.

The user can continue performing operations until Exit is selected.

21. PRACTICAL CASE STUDY SCENARIO

21.1 Scenario Introduction

Consider a fictional patient named Aarav Sharma visiting MediCore General Hospital.

He requires a general medical consultation.

The reception staff must register his information and arrange an appointment with an available doctor.

21.2 Patient Registration

The reception staff enters:

Field	Value
Patient ID	101
Name	Aarav Sharma
Age	20
Contact	9876501234
Condition	Fever (demonstration)
Priority	3

The system checks whether the patient ID already exists.

After successful validation, the application creates the patient record.

21.3 Doctor Registration

A doctor named Neha Kapoor is registered.

Field	Value
Doctor ID	201
Name	Neha Kapoor
Age	38
Contact	9876501111
Specialization	General Medicine

The application creates the Doctor object and stores it.

21.4 Appointment Scheduling

The reception staff selects patient ID 101 and doctor ID 201.

The application checks both records.

Since both exist and the doctor is marked available, the appointment can be scheduled.

21.5 Treatment Record Creation

Following the hypothetical consultation, the operator creates a demonstration treatment record associated with patient 101 and doctor 201.

21.6 Administrative Review

The administrator searches for the patient and views the available records.

The administrator can also access the statistics module to review the current record totals.

21.7 Scenario Outcome

This scenario demonstrates how the Patient, Doctor, Appointment, Treatment, and Statistics modules work together.

It shows how the application organizes a basic administrative workflow through separate but related components.

IMPLEMENTATION ENVIRONMENT AND DATA MANAGEMENT

22. IMPLEMENTATION ENVIRONMENT

22.1 Programming Language

The application is developed using C++.

C++ is suitable for this project because it supports classes, objects, inheritance, encapsulation, polymorphism, and standard library containers.

22.2 Development Tools

Component	Technology
Programming Language	C++
Compiler	GCC / Clang / MSVC
Language Standard	C++17
Code Editor	Visual Studio Code or equivalent
Application Interface	Console
Data Storage	In-memory vectors
Database	Not used
External Backend	Not used

22.3 Hardware Requirements

The program can run on a standard desktop or laptop computer.

Required hardware includes a computer with a compatible operating system, keyboard, display, and sufficient memory to execute a basic C++ application.

22.4 Software Requirements

A C++17-compatible compiler is required.

The program uses standard C++ library headers and does not require third-party libraries.

23. DATA MANAGEMENT AND RECORD ORGANIZATION

23.1 Introduction

MediCore uses in-memory collections to maintain records.

Each collection stores objects belonging to a particular class.

23.2 Patient Collection

The patient collection stores all registered Patient objects.

Each patient is identified by a unique patient ID within the collection.

23.3 Doctor Collection

The doctor collection stores Doctor objects.

Each doctor is identified by a unique doctor ID within the collection.

23.4 Appointment Collection

The appointment collection stores appointment information, including appointment ID, patient ID, doctor ID, date/time, and status.

23.5 Treatment Collection

The treatment collection stores patient ID, doctor ID, diagnosis text, and treatment notes.

23.6 Data Validation

The application performs basic validation before creating records.

Examples include:

- Duplicate patient ID checking.
- Duplicate doctor ID checking.
- Positive age checking.
- Priority range checking.
- Patient ID verification.
- Doctor ID verification.
- Empty text-field checking.

23.7 Storage Limitations

The application does not use a permanent database or file-storage mechanism.

All records remain available only during the current execution.

When the program closes, the records are lost.

24. TESTING AND VERIFICATION PLAN

24.1 Introduction

Testing is an important part of software development.

It helps identify errors and verifies whether the application performs its intended operations.

MediCore should be tested using valid information, invalid information, missing IDs, duplicate records, and empty collection scenarios.

24.2 Functional Testing

Functional testing checks whether individual application features behave according to their requirements.

The following test cases should be executed.

Test ID	Test Scenario	Expected Result
TC-01	Register valid patient	Patient registered
TC-02	Register duplicate patient ID	Registration rejected
TC-03	Enter invalid patient age	Invalid record rejected
TC-04	Enter priority outside 1–5	Registration rejected
TC-05	Search existing patient	Details displayed
TC-06	Search missing patient	Not found message
TC-07	Register valid doctor	Doctor registered
TC-08	Register duplicate doctor ID	Registration rejected
TC-09	Schedule valid appointment	Appointment created
TC-10	Enter invalid patient ID	Appointment cancelled
TC-11	Enter invalid doctor ID	Appointment cancelled
TC-12	Add valid treatment record	Record created
TC-13	Add treatment with invalid IDs	Record rejected
TC-14	Display empty appointment list	Empty-list message
TC-15	Display empty treatment list	Empty-list message

TC-16	Run priority simulation	Smallest configured waiting priority selected
TC-17	Generate statistics	Correct totals displayed
TC-18	Enter alphabetic input at numeric prompt	Input error handled
TC-19	Select invalid menu option	Error message displayed
TC-20	Exit application	Program terminates

24.3 Testing Methodology

The testing process should follow these steps:

Step 1: Compile the program using a C++17-compatible compiler.

Step 2: Start the application and verify that the main menu appears.

Step 3: Register sample patient and doctor records.

Step 4: Search for existing and missing patients.

Step 5: Schedule appointments using valid and invalid IDs.

Step 6: Add and display treatment records.

Step 7: Run the emergency-priority simulation.

Step 8: Generate hospital statistics.

Step 9: Test invalid inputs and duplicate IDs.

Step 10: Exit the application and verify normal termination.

24.4 Expected Testing Outcome

The application should perform the specified operations and display appropriate success or error messages.

Actual test results should be recorded after executing the program.

24.5 Testing Evidence

Screenshots should be captured directly from the running program.

Recommended screenshots include:

1. Main menu.
2. Patient registration.
3. Patient searching.
4. Doctor registration.
5. Appointment scheduling.

6. Treatment record creation.
7. Emergency simulation.
8. Hospital statistics.
9. Invalid input handling.

These screenshots can be included in the final report as implementation evidence.

SECURITY, ADVANTAGES, LIMITATIONS AND FUTURE SCOPE

25. SECURITY AND PRIVACY CONSIDERATIONS

25.1 Confidentiality

Patient information should be treated as sensitive information.

The current application does not implement authentication or access restrictions.

25.2 Data Integrity

Basic validation is performed to reduce duplicate identifiers and invalid entries.

However, the application does not implement comprehensive data-integrity controls.

25.3 Data Availability

Since information is stored in memory, records are unavailable after program termination.

25.4 Access Control

The application does not provide separate access permissions for administrators, reception staff, or doctors.

25.5 Privacy Considerations

Only fictional information should be used during project testing.

A real healthcare application would require appropriate security, privacy safeguards, access control, secure storage, and compliance with applicable regulations.

26. ADVANTAGES OF THE PROPOSED SYSTEM

1. Centralized Operations: Different administrative operations are available through one application.

2. Organized Information: Separate classes maintain different categories of information.

3. Patient Searching: Patient records can be searched using unique identification numbers.

4. Appointment Coordination: The system verifies required patient and doctor records before scheduling.

5. Basic Validation: The application checks several invalid entries.

6. Modular Structure: Separate classes and methods improve code organization.

7. Automatic Statistics: Record totals can be calculated without manual counting.

8. Educational Value: The project demonstrates practical applications of C++ and object-oriented design.

9. No External Dependencies: The application does not require an external database or third-party libraries.

27. LIMITATIONS OF THE PROPOSED SYSTEM

The current implementation has several limitations.

- It does not provide permanent record storage.
- It does not include a graphical user interface.
- It does not integrate with a database.
- It does not implement authentication.
- It does not detect appointment time conflicts.
- It does not provide advanced reporting.
- It does not include a real clinical workflow.
- Its emergency-priority mechanism is only a simplified educational simulation.

28. FUTURE SCOPE

MediCore can be enhanced through several additional features.

File Handling: Records could be saved in local files and loaded when the application starts.

Advanced Appointment Scheduling: The system could validate dates, prevent scheduling conflicts, and support cancellation or rescheduling.

Doctor Availability: Doctor working hours and shifts could be maintained.

Enhanced Patient Records: Additional administrative information could be included.

Advanced Reporting: More detailed reports could be generated.

Improved Validation: Contact information and date formats could be validated more thoroughly.

Audit History: A future version could record important changes made to the system.

Enhanced Security: Authentication, authorization, secure storage, and privacy controls would be necessary for a real application.

Clinical Integration: Any clinical functionality would require qualified medical professionals, validated protocols, and appropriate safety review.

CONCLUSION AND REFERENCES

29. CONCLUSION

MediCore — Hospital Emergency and Patient Management System is a proposed C++-based application designed to organize basic hospital-management activities through a structured and modular approach.

The case study identifies common administrative challenges associated with maintaining patient information, doctor records, appointments, and treatment documentation.

It proposes a centralized application in which different hospital entities are represented through separate classes and managed using a menu-driven interface.

The proposed system includes patient registration, doctor registration, patient searching, appointment scheduling, treatment record management, emergency-priority simulation, and hospital statistics.

The application demonstrates how object-oriented programming can be applied to a practical management scenario.

It also provides an opportunity to understand the importance of modular design, structured information management, basic validation, and software testing.

The project is developed entirely in C++ and does not require an external database, graphical user interface, or separate backend.

Although the current version has limitations such as the absence of permanent storage, authentication, and advanced appointment management, it provides a foundation for future development.

The case study demonstrates that programming concepts can be applied to organize real-world administrative activities and that structured software design can help make applications easier to understand, maintain, and extend.

Overall, MediCore provides valuable practical experience in analyzing a problem, identifying requirements, designing a solution, implementing functional modules, and evaluating the expected performance of a software application.

30. REFERENCES

The following resources can be consulted during project development and documentation:

1. Bjarne Stroustrup, *The C++ Programming Language*.
2. C++ Standard Library documentation.
3. C++ language reference and learning resources.
4. Object-Oriented Programming course material.

5. Classroom notes on classes, objects, inheritance, encapsulation, abstraction, and polymorphism.
6. General educational resources on hospital information management.
7. Software engineering and system-analysis study material.